

深圳大学实验报告

课程名称：移动互联网应用

实验项目名称：课堂练习 2

学院：计算机与软件学院

专业：软件工程

指导教师：解为成

报告人：__学号：__班级：__

实验时间：2026 年 4 月 20 日

实验报告提交时间：2026 年 4 月 20 日

教务部制

实验目的与要求：

怎样实现两个碎片之间的通信？

（书本 P151，以活动作为媒介建立两个碎片之间的通信）

注意：

1. 实验报告中需要有实验结果的截屏图像；
2. 尽量删除空白页。

方法、步骤：

1. 基于教材 P151 页的理论，由于 Fragment 之间不能直接建立显式的引用关联，因此采用以 Activity 作为通信媒介的方法。

具体路径为：发送方 Fragment → 获取宿主 Activity → 获取接收方 Fragment 实例 → 调用公开方法传递数据。

2. **界面布局设计。**创建宿主 Activity 布局（横向平分加载左右两个 Fragment），并为右侧 Fragment 设定固定 ID（R.id.right_fragment）以供寻址；分别创建左侧包含 Button 的布局和右侧包含 TextView 的布局。

3. **构建接收方（RightFragment）。**在 Java 类中绑定 TextView 控件，并对外暴露一个公共方法 updateText(String text)，用于接收外部传入的字符串并更新 UI。

4. **构建发送方（LeftFragment）。**在 Java 类中绑定 Button 并设置点击监听器。在监听器内部实现核心通信逻辑：通过 getActivity() 获取宿主 Activity，结合 getSupportFragmentManager().findFragmentById() 找到 RightFragment 实例。

5. **配置 AndroidManifest.xml 文件。**为 MainActivity 添加 MAIN 和 LAUNCHER 的 Intent 过滤器，使其成为 App 的默认启动页。

6. **编译与运行测试。**将程序部署至 Android 虚拟机或真机，进行点击交互测试验证功能。

实验过程及内容:

1. 静态加载与布局装配:

在 activity_main.xml 中使用 LinearLayout 将屏幕水平分为两部分，利用 `<androidx.fragment.app.FragmentContainerView>` 引入 LeftFragment 和 RightFragment。关键操作：必须给右侧 Fragment 容器指定属性 `android:id="@+id/right_fragment"`，这是后续查找通信目标的关键标识。

```
© RightFragment.java  </> activity_main.xml x  </> fragment_left.xml  </> fragment_right.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:orientation="horizontal">
6
7      <!-- 静态加载左侧碎片 -->
8      <androidx.fragment.app.FragmentContainerView
9          android:id="@+id/left_fragment"
10         android:name="com.example.test2.LeftFragment"
11         android:layout_width="0dp"
12         android:layout_height="match_parent"
13         android:layout_weight="1" />
14
15     <!-- 静态加载右侧碎片，设置ID以便寻找 -->
16     <androidx.fragment.app.FragmentContainerView
17         android:id="@+id/right_fragment"
18         android:name="com.example.test2.RightFragment"
19         android:layout_width="0dp"
20         android:layout_height="match_parent"
21         android:layout_weight="1" />
22
23 </LinearLayout>
```

activity_main.xml

```
© RightFragment.java  </> activity_main.xml  </> fragment_left.xml x  </> fragment_right.xml
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent"
5      android:background="#E8F5E9"
6      android:gravity="center"
7      android:orientation="vertical">
8
9      <Button
10         android:id="@+id/btn_send_message"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="发送消息给右边" />
14
15 </LinearLayout>
```

fragment_left.xml

```
© RightFragment.java </> activity_main.xml </> fragment_left.xml </> fragment_right.xml x
1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:background="#E3F2FD"
6     android:gravity="center"
7     android:orientation="vertical">
8
9     <TextView
10         android:id="@+id/tv_receive_message"
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="等待接收消息..."
14         android:textSize="20sp" />
15
16 </LinearLayout>
```

fragment_right.xml

2. 通信接收端的实现:

编写 RightFragment.java, 在其 onCreateView 生命周期中完成视图的加载和控件的初始化。

```
activity.java AndroidManifest.xml © LeftFragment.java © RightFragment.java x </> activity_main.xml </> fragment_left.xml </> fragment_right.xml v
1 package com.example.test2;
2
3 import android.os.Bundle;
4 import android.view.LayoutInflater;
5 import android.view.View;
6 import android.view.ViewGroup;
7 import android.widget.TextView;
8 import androidx.annotation.NonNull;
9 import androidx.annotation.Nullable;
10 import androidx.fragment.app.Fragment;
11
12 </> public class RightFragment extends Fragment {
13
14     3 usages
15     private TextView tvReceiveMessage;
16
17     @Nullable
18     @Override
19     public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
20         // 加载右边碎片的布局
21         View view = inflater.inflate(R.layout.fragment_right, container, attachToRoot: false);
22         // 绑定 TextView 控件
23         tvReceiveMessage = view.findViewById(R.id.tv_receive_message);
24         return view;
25     }
26
27     /**
28      * 这是一个公开(public)的方法, 供左边碎片通过Activity调用
29      */
30     no usages
31     public void updateText(String text) {
32         if (tvReceiveMessage != null) {
33             tvReceiveMessage.setText(text);
34         }
35     }
36 }
```

RightFragment.java

3. 通信发送端及中转逻辑的实现：

编写 LeftFragment.java，在按钮的 onClick 事件中，按照教材理论编写了中转通信代码：

```
activity.java  AndroidManifest.xml  LeftFragment.java  RightFragment.java  activity_main.xml  fragment_left.xml  fragment_right.xml
12 public class LeftFragment extends Fragment {
13
14     @Nullable
15     @Override
16     public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
17         // 加载左边碎片的布局
18         View view = inflater.inflate(R.layout.fragment_left, container, attachToRoot: false);
19
20         Button btnSendMessage = view.findViewById(R.id.btn_send_message);
21         btnSendMessage.setOnClickListener(new View.OnClickListener() {
22             @Override
23             public void onClick(View v) {
24                 // ===== 书本 P151 的核心通信逻辑 =====
25
26                 // 1. 获取当前碎片所在的 Activity 实例
27                 MainActivity activity = (MainActivity) getActivity();
28
29                 if (activity != null) {
30                     // 2. 通过 Activity 的 FragmentManager 去寻找右边碎片的实例
31                     // (注意：这里使用 getSupportFragmentManager() 替代老版本的 getFragmentManager())
32                     RightFragment rightFragment = (RightFragment) activity.getSupportFragmentManager().findFragmentById(R.id.right_fragment);
33
34                     // 3. 找到右边碎片后，直接调用它的 public 方法传递数据
35                     if (rightFragment != null) {
36                         rightFragment.updateText("你好！我是左边碎片发来的消息。");
37                     }
38                 }
39             }
40         });
41
42         return view;
43     }
44 }
45 }
```

LeftFragment.java

4. 配置 AndroidManifest.xml 文件：

为 MainActivity 添加 MAIN 和 LAUNCHER 的 Intent 过滤器，使其成为 App 的默认启动页。

```
activity.java  AndroidManifest.xml  LeftFragment.java  RightFragment.java  activity_main.xml  fragment_left.xml  fragment_right.xml
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.test2">
4
5     <application
6         android:allowBackup="true"
7         android:icon="@mipmap/ic_launcher"
8         android:label="@string/app_name"
9         android:roundIcon="@mipmap/ic_launcher_round"
10        android:supportRtl="true"
11        android:theme="@style/Theme.Test2"> <!-- 你的主题名可能略有不同，保留原来的即可 -->
12
13        <!-- ===== 核心修改在这里 ===== -->
14        <activity
15            android:name=".MainActivity"
16            android:exported="true"> <!-- 注意：exported="true" 在新版本Android中是必须的 -->
17
18            <intent-filter>
19                <!-- 声明这是App的入口点 -->
20                <action android:name="android.intent.action.MAIN" />
21                <!-- 声明在手机桌面上显示图标 -->
22                <category android:name="android.intent.category.LAUNCHER" />
23            </intent-filter>
24
25        </activity>
26        <!-- ===== -->
27
28    </application>
29
30 </manifest>
```

AndroidManifest.xml

数据处理分析：

1. 用户点击 `LeftFragment` 上的按钮，触发事件，内存中生成了硬编码的字符串数据 "你好！我是左边碎片发来的消息。"
2. 由于左侧无法直接看到右侧，数据必须向上层抛出。`LeftFragment` 利用自身持有的宿主引用 (`getActivity()`)，将操作权限转移给了 `MainActivity` 的管理器 (`FragmentManager`)。
3. `FragmentManager` 像一个路由表，通过比对布局文件中定义的 `R.id.right_fragment`，在内存中精准匹配到了已经实例化的 `RightFragment` 对象。
4. 字符串数据作为参数传入 `updateText()` 方法，最终交由 Android 的 UI 线程渲染到 `TextView` 上，完成了从左到右的数据传递。

实验结论：



App 启动初始状态，右侧数据显示为“等待接收消息...”。点击左侧按钮后，状态立即刷新为预设的传递内容。数据状态的无延迟更替，从表现层证明了通过 Activity 媒介实现 Fragment 间跨域调用方法的通信模型是完全正确且高效的。

指导教师批阅意见：

成绩评定：

指导教师签字：解为成
2026 年 4 月 20 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。